

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE NAME: MLA03**

**COURSE CODE: Reinforcement learning**

---

### **COURSE OUTCOME COVERED**

**CO1:** *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

*Assessment Tool Weightage: 50%*

---

Dr. G. A. Senthil

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

**RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator**

**Duration: 2hrs**

#### **Experiment 1: Installation of Reinforcement Learning Development Environment**

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

#### **Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments**

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

#### **Experiment 3: Implementation of the Reinforcement Learning Framework**

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

#### **Experiment 4: Simulation of a Markov Decision Process (MDP)**

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

### **Experiment 5: Bellman Equation Demonstration**

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

### **Experiment 6: Exploration vs. Exploitation Strategies**

Aim: To implement  $\epsilon$ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

### **Experiment 7: Multi-Armed Bandit Problem**

Aim: To implement the Multi-Armed Bandit problem using  $\epsilon$ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

### **Experiment 8: Reward Visualization in Reinforcement Learning**

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

### **Experiment 9: TensorFlow and Keras-Based Neural Network for RL**

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

### **Experiment 10: Mini Reinforcement Learning Project Using CartPole**

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

---

#### ***Code of Conduct:***

*I K.Shalini(192472213) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

#### **RUBRICS FOR CALCULATION**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                  | <b>Level 3 – Proficient</b>             | <b>Level 2 – Developing</b>                  | <b>Level 1 – Beginning</b>               |
|---------------------------|------------------|-------------------------------------------------------------|-----------------------------------------|----------------------------------------------|------------------------------------------|
| Understanding of Concepts | 25%              | Demonstrates thorough, accurate understanding of concepts   | Good understanding with minor gaps      | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation               | 25%              | Detailed, well-explained answers with relevant examples     | Clear explanation with adequate details | Limited explanation; lacks depth             | Poor or unclear explanation              |
| Application               | 20%              | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors      | Limited application with weak analysis       | No application or incorrect analysis     |
| Organization              | 15%              | Well-structured answers with logical flow and coherence     | Organized with minor issues in flow     | Basic structure, lacks clarity               | Poor organization, incoherent answers    |
| Accuracy                  | 10%              | Completely accurate and covers all required points          | Mostly accurate with minor omissions    | Partially correct with missing points        | Incorrect answers with major gaps        |
| Clarity                   | 5%               | Clear, neat, professional presentation                      | Understandable with minor issues        | Limited clarity, readability issues          | Poorly written, difficult to understand  |

## Experiment – 01

### Installation of Reinforcement Learning Development Environment

#### Program :

```
import gymnasium as gym

env = gym.make("CartPole-v1")

observation, info = env.reset()

print("Environment created successfully.")

print("Initial Observation:", observation)

env.close()
```

#### Output :

```
===== RESTART: C:/Users/DELL/Downloads/Experiment - 01.py =====
Environment created successfully.
Initial Observation: [-0.01084141  0.01977365  0.00314763  0.01211106]
|
```

## Experiment 2

### Familiarization with OpenAI Gym/Gymnasium Environments

#### Program :

```
import gymnasium as gym

environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]

for env_name in environments:

    print("\n-----")

    print("Environment:", env_name)

    env = gym.make(env_name)

    observation, info = env.reset()

    print("Initial State:", observation)

    print("Number of Actions:", env.action_space)
```

```

action = env.action_space.sample()

observation, reward, terminated, truncated, info = env.step(action)

print("Random Action:", action)

print("Next State:", observation)

print("Reward:", reward)

print("Episode Finished:", terminated or truncated)

env.close()

```

### Sample Output :

```

===== RESTART: C:/Users/DELL/Downloads/Experiment - 2.py =====

-----
Environment: FrozenLake-v1
Initial State: 0
Number of Actions: Discrete(4)
Random Action: 2
Next State: 0
Reward: 0
Episode Finished: False

-----
Environment: CartPole-v1
Initial State: [ 0.04215307 -0.04296664 -0.00168217 -0.03618058]
Number of Actions: Discrete(2)
Random Action: 0
Next State: [ 0.04129374 -0.23806442 -0.00240578  0.25597113]
Reward: 1.0
Episode Finished: False

-----
Environment: MountainCar-v0
Initial State: [-0.5269484  0.          ]
Number of Actions: Discrete(3)
Random Action: 0
Next State: [-0.5279233  -0.00097488]
Reward: -1.0
Episode Finished: False
>>>

```

### Experiment 03 :

#### Implementation of the Reinforcement Learning Framework

##### Program :

```

import gymnasium as gym

```

```
env = gym.make("FrozenLake-v1", is_slippery=False)

state, info = env.reset()

print("Initial State:", state)

total_reward = 0

for step in range(10):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Step:", step + 1)

    print("Current State:", state)

    print("Action:", action)

    print("Next State:", next_state)

    print("Reward:", reward)

    total_reward += reward

    state = next_state

    if terminated or truncated:

        print("Episode Finished")

        break

print("Total Reward:", total_reward)

env.close()
```

**Sample Output :**

```

===== RESTART: C:/Users/DELL/Downloads/Experiment - 3.py =====
Initial State: 0
Step: 1
Current State: 0
Action: 1
Next State: 4
Reward: 0
Step: 2
Current State: 4
Action: 0
Next State: 4
Reward: 0
Step: 3
Current State: 4
Action: 1
Next State: 8
Reward: 0
Step: 4
Current State: 8
Action: 1
Next State: 12
Reward: 0
Episode Finished
Total Reward: 0

```

## Experiment 04 :

### Simulation of a Markov Decision Process (MDP)

#### Program :

```
states = ["Start", "Middle", "Goal"]
```

```
rewards = {
```

```
    "Start": 0,
```

```
    "Middle": 5,
```

```
    "Goal": 10
```

```
}
```

```
current_state = "Start"
```

```
print("Initial State:", current_state)
```

```
while current_state != "Goal":
```

```
    if current_state == "Start":
```

```

        next_state = "Middle"

    elif current_state == "Middle":

        next_state = "Goal"

    print("Current State:", current_state)

    print("Next State:", next_state)

    print("Reward:", rewards[next_state])

    current_state = next_state

print("Final State:", current_state)

print("Process Completed")

```

### Sample Output :

```

>>> ===== RESTART: C:/Users/DELL/Downloads/Experiment - 4.py =====
Initial State: Start
Current State: Start
Next State: Middle
Reward: 5
Current State: Middle
Next State: Goal
Reward: 10
Final State: Goal
Process Completed

```

## Experiment 05 :

### Bellman Equation Demonstration

#### Program :

```

gamma = 0.9

rewards = [0, 5, 10]

values = [0, 0, 0]

print("Initial State Values:", values)

for iteration in range(5):

    new_values = values.copy()

```



```

for i in range(len(values) - 1):
    new_values[i] = rewards[i] + gamma * values[i + 1]
new_values[2] = rewards[2]
values = new_values
print("Iteration", iteration + 1, ":", values)
print("Final State Values:", values)

```

### Sample Output :

```

>>> |===== RESTART: C:/Users/DELL/Downloads/Experiment - 5.py =====
      Initial State Values: [0, 0, 0]
      Iteration 1 : [0.0, 5.0, 10]
      Iteration 2 : [4.5, 14.0, 10]
      Iteration 3 : [12.6, 14.0, 10]
      Iteration 4 : [12.6, 14.0, 10]
      Iteration 5 : [12.6, 14.0, 10]
      Final State Values: [12.6, 14.0, 10]
>>> |

```

## Experiment 06 :

### Exploration vs. Exploitation Strategies

#### Program :

```

import random

action_values = [5, 10, 8]

epsilon = 0.2

greedy_action = action_values.index(max(action_values))

random_action = random.randint(0, len(action_values) - 1)

if random.random() < epsilon:

    epsilon_action = random.randint(0, len(action_values) - 1)

else:

    epsilon_action = greedy_action

print("Action Values:", action_values)

print("Greedy Action:", greedy_action)

```

```
print("Random Action:", random_action)

print("Epsilon-Greedy Action:", epsilon_action)
```

### Sample Output :

```
>>> |===== RESTART: C:/Users/DELL/Downloads/Experiment - 6.py =====
      |Action Values: [5, 10, 8]
      |Greedy Action: 1
      |Random Action: 1
      |Epsilon-Greedy Action: 1
>>> |
```

## Experiment 07 :

### Multi-Armed Bandit Problem

#### Program :

```
import random

import math

rewards = [2, 5, 8]

epsilon = 0.2

greedy_reward = 0

ucb_reward = 0

counts = [1, 1, 1]

for i in range(10):

    if random.random() < epsilon:

        action = random.randint(0, 2)

    else:

        action = rewards.index(max(rewards))

    greedy_reward += rewards[action]

    ucb_values = []
```

```

for j in range(3):

    value = rewards[j] + math.sqrt((2 * math.log(i + 2)) / counts[j])

    ucb_values.append(value)

ucb_action = ucb_values.index(max(ucb_values))

ucb_reward += rewards[ucb_action]

counts[ucb_action] += 1

print("Total Reward using Epsilon-Greedy:", greedy_reward)

print("Total Reward using UCB:", ucb_reward)

```

### Sample Output :

```

>>> |===== RESTART: C:/Users/DELL/Downloads/Experiment - 7.py ==
      |Total Reward using Epsilon-Greedy: 80
      |Total Reward using UCB: 80
>>> |

```

---

## Experiment 08 :

### Reward Visualization in Reinforcement Learning

#### Program :

```

import matplotlib.pyplot as plt

episode_rewards = [1, 2, 3, 2, 4, 5, 6, 5, 7, 8]

cumulative_rewards = []

total = 0

for reward in episode_rewards:

    total += reward

    cumulative_rewards.append(total)

plt.plot(cumulative_rewards, marker='o')

```

```
plt.title("Cumulative Rewards")
```

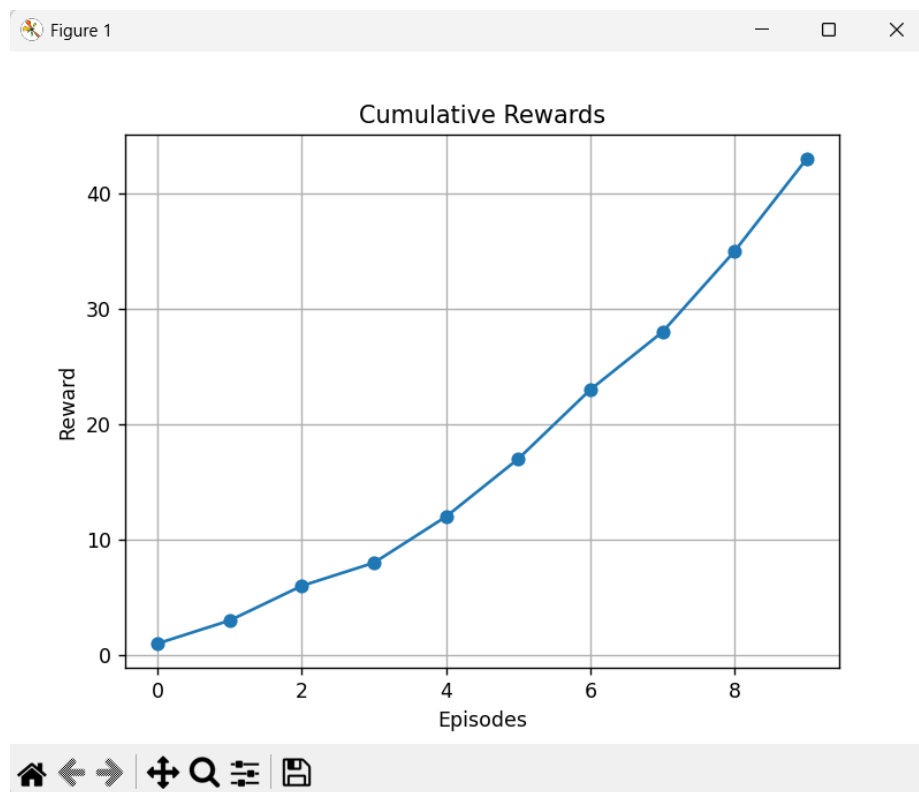
```
plt.xlabel("Episodes")
```

```
plt.ylabel("Reward")
```

```
plt.grid(True)
```

```
plt.show()
```

**Sample Output :**



**Experiment 09 :**

**TensorFlow and Keras-Based Neural Network for RL**

**Program :**

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```

from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(24, activation="relu", input_shape=(4,)))
model.add(Dense(24, activation="relu"))
model.add(Dense(2, activation="linear"))
model.compile(optimizer="adam", loss="mse")
model.summary()

```

### Sample Output :

```

>>> | ===== RESTART: C:/Users/DELL/Downloads/Experiment - 9.py =====
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Ple
ase use WSL2 or the TensorFlow-DirectML plugin.
Model: "sequential"

```

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 24)   | 120     |
| dense_1 (Dense) | (None, 24)   | 600     |
| dense_2 (Dense) | (None, 2)    | 50      |

```

Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)
>>> |

```

## Experiment 10 :

### Mini Reinforcement Learning Project Using CartPole

#### Program :

```

import gymnasium as gym
env = gym.make("CartPole-v1")
state, info = env.reset()
total_reward = 0
done = False
while not done:
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward

```

```
done = terminated or truncated
print("Total Reward:", total_reward)
env.close()
```

### Sample Output :

```
>>> |
===== RESTART: C:/Users/DELL/Downloads/Experiment 10.py =====
Total Reward: 20.0
>>> |
```

---